ozontech
образование

Поиск CodeSmells. Концепция автономных агентов.



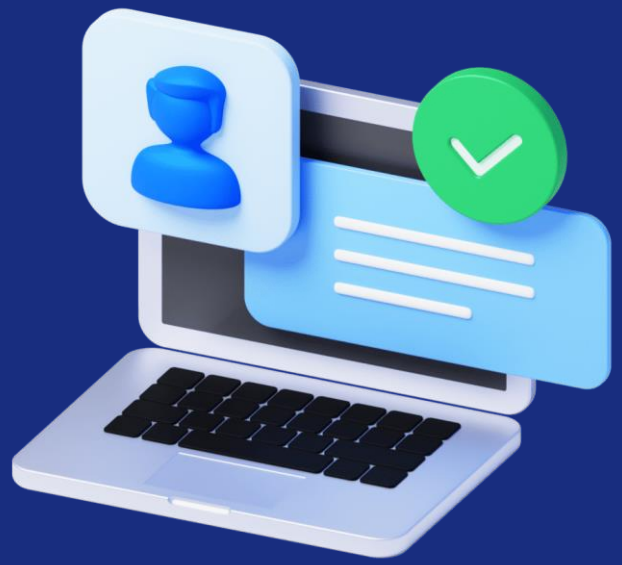
Привет!

Я Кузьменко Иван

Senior AQA в Ozon Tech ML Platform

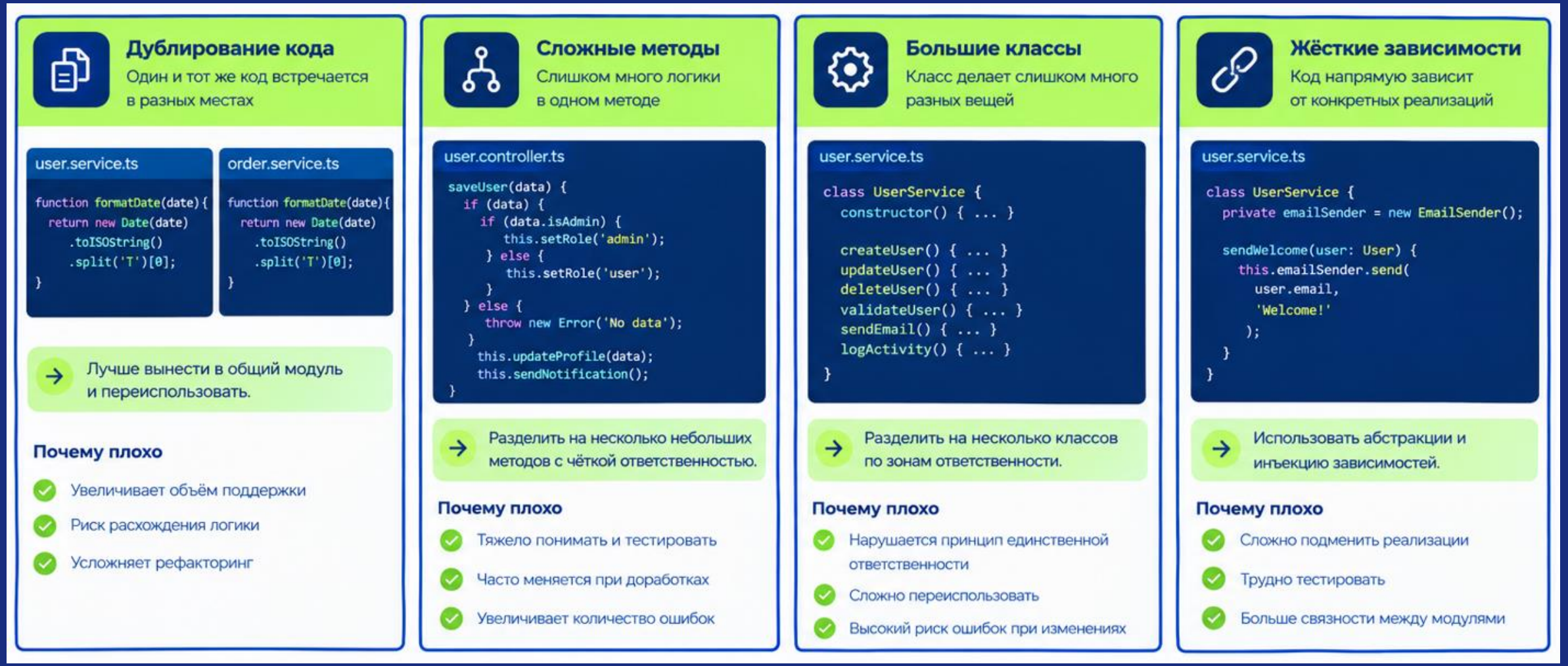


11:25



Рефакторинг, оптимизация и миграция legacy-кода

Код с запашком



ВНИМАНИЕ

Модели обучены на открытом коде. Большинство кода написано плохо

11:25



LLM-агент для поиска Code Smells и уязвимостей

Идея	Проблемы	Решение
- Промпт-инжиниринг вместо сложных систем анализа. - Дешёвый способ выявления проблем.	- Воспроизводимость результатов - Агенты плохо видят архитектурные проблемы	- Сформулировать метрику - Дать инструменты: AST-парсеры, линтеры, SAST, профайлеры

Пример A: Поиск Code Smells

Что ищем

- Нарушения SOLID, DRY, KISS
- God Objects, длинные методы, tight coupling

Инструменты

- Tree-sitter (AST-парсер)
- Radon (Python) / lizard (мультиязычный)

Пример B: Security Audit

Что ищем

- SQL-инъекции, XSS, SSRF
- Утечки секретов, race conditions

Инструменты

- trufflehog / git-leaks
- Bandit, Semgrep, Snyk

Пример C: Performance Bottlenecks

Что ищем

- Неэффективные алгоритмы (O(N²) вместо O(N))
- N+1 запросы, отсутствие индексов

Инструменты

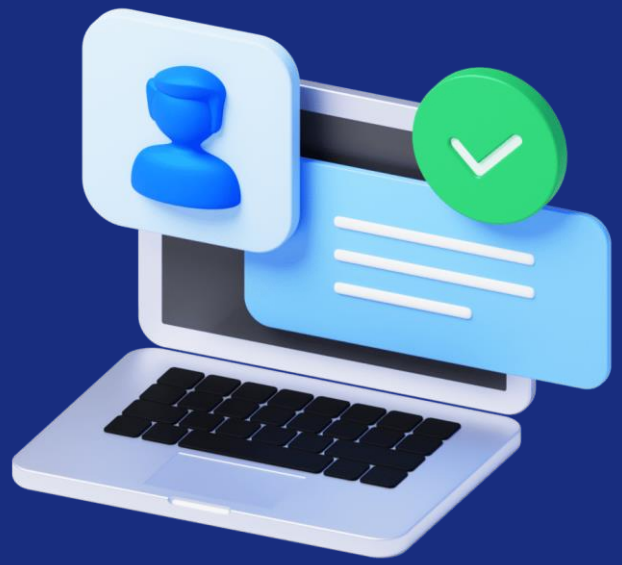
- cProfile (Python) / pprof (Go)
- EXPLAIN ANALYZE на тестовой БД

ВНИМАНИЕ

Хороший код – поддерживаемый код



11:25



Крупный рефакторинг. Миграция кода

Рефакторинг крупных кодовых баз

Цель

Автоматически исправить техдолг, декомпозировать легаси, обновить библиотеки без поломки поведения.

Гарантия

- Тесты: агент запускает тесты сам
- Принцип «Нет ТЗ — результат ХЗ»: метрики вместо слов

Миграция кода между языками

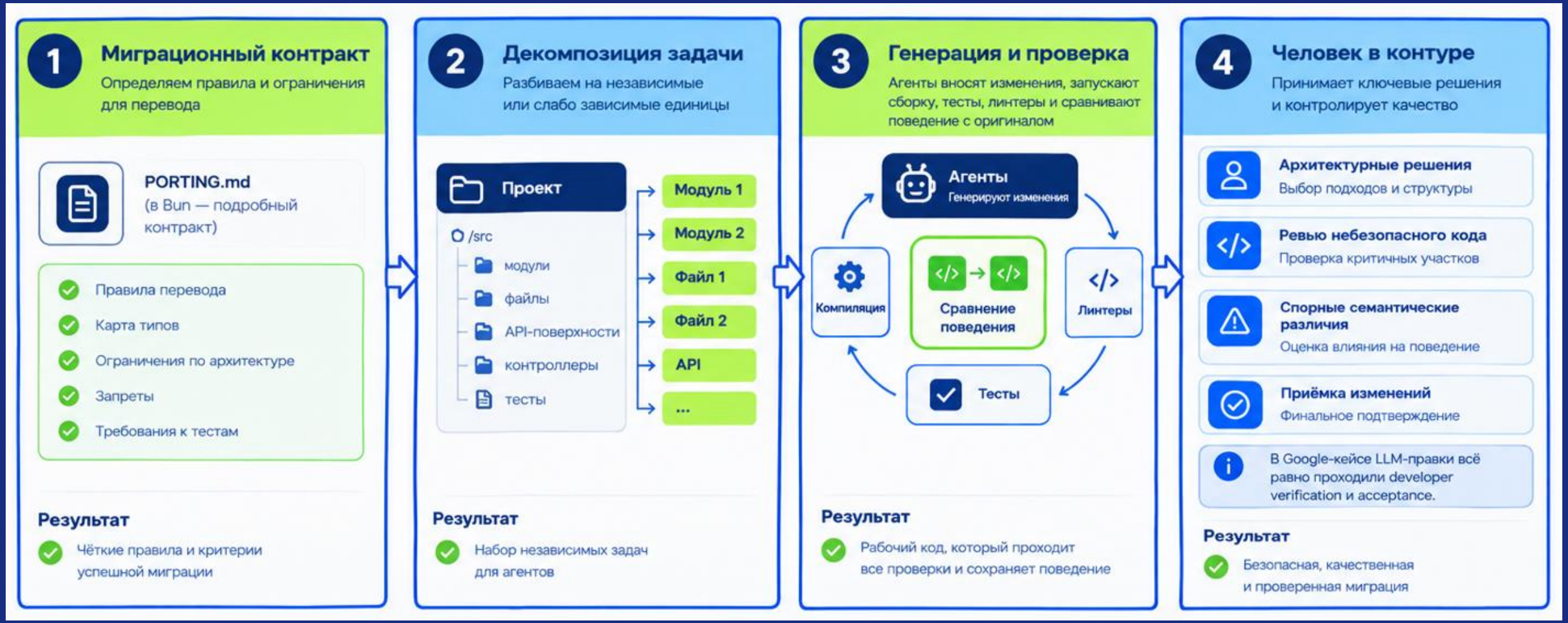
Задача

Перенести логику со старого языка на новый стек, сохранив поведение.

Приёмы

- Дифференциальное тестирование: e2e + fuzzing
- Двухэтапный трансляционный промптинг: спецификации → код

Цикл миграции

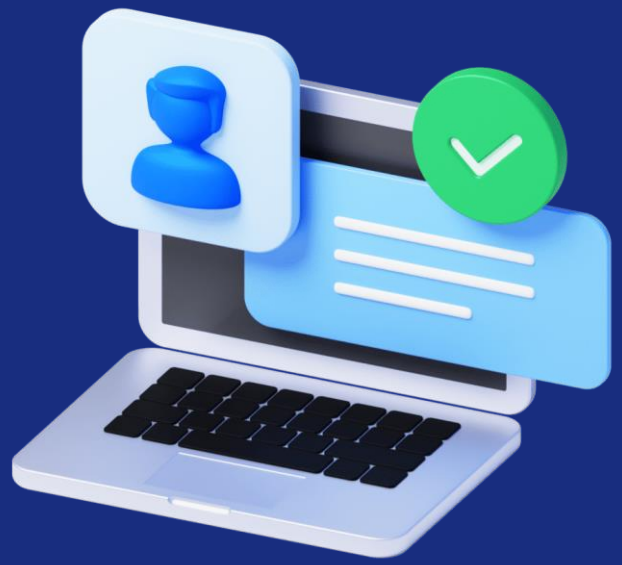


ВНИМАНИЕ

Агентно-управляемая миграция с жёсткой инженерной обвязкой



11:25



Мультиагентные системы. Автономные агенты

Single-Agent vs Multi-Agent

Single-Agent

Один агент решает задачу последовательно.

Multi-Agent

Разные роли специализированных агентов: общаются, рецензируют код, координируют.

Разные роли

Минималистичная, но мощная модель



Explore (read-only) — Быстрый разведчик кодовой базы. Что существует?

- поиск файлов
- code search
- анализ структуры
- уровень thoroughness: quick / medium / very thorough



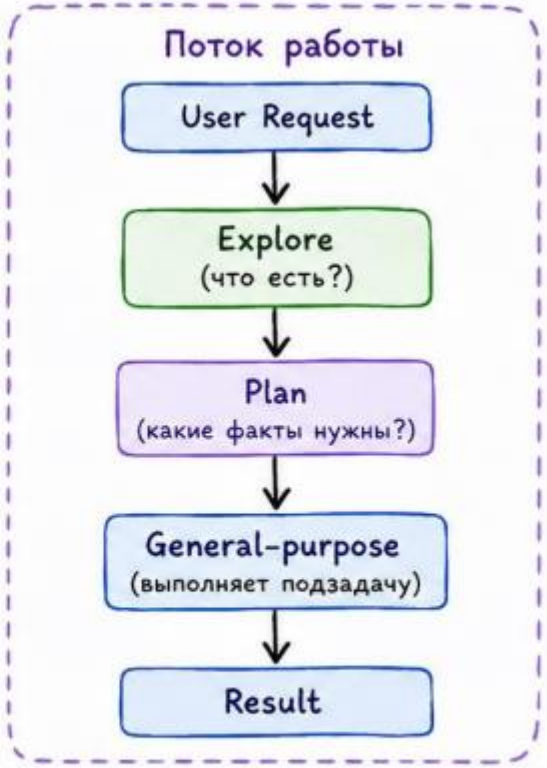
Plan (read-only) — Сбор фактов для построения плана. Какие факты нужны перед решением?

- фокус на планировании
- факты и зависимости
- не выполняет изменение кода



General-purpose (worker) — Полноценный исполнитель. Режим подзадачу

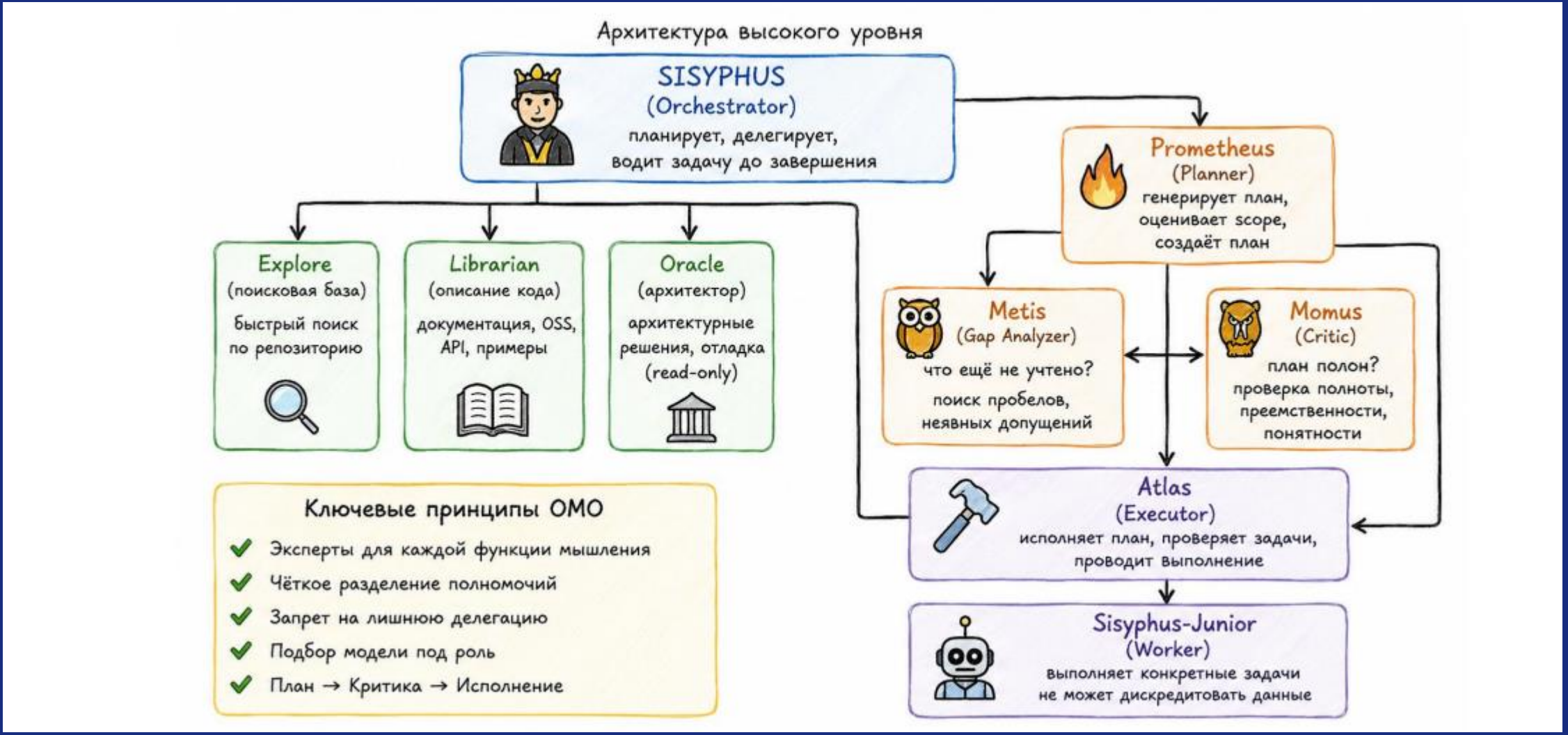
- исследование
- рассуждение
- выполнение действий
- изменение кода



Важно:

- ✓ Explore и Plan — one-shot, не могут быть продолжены
- ✓ Не имеют прав Write / Edit
- ✓ Предназначены для исследования и подготовки контекста

OpenCode + plugin OMO



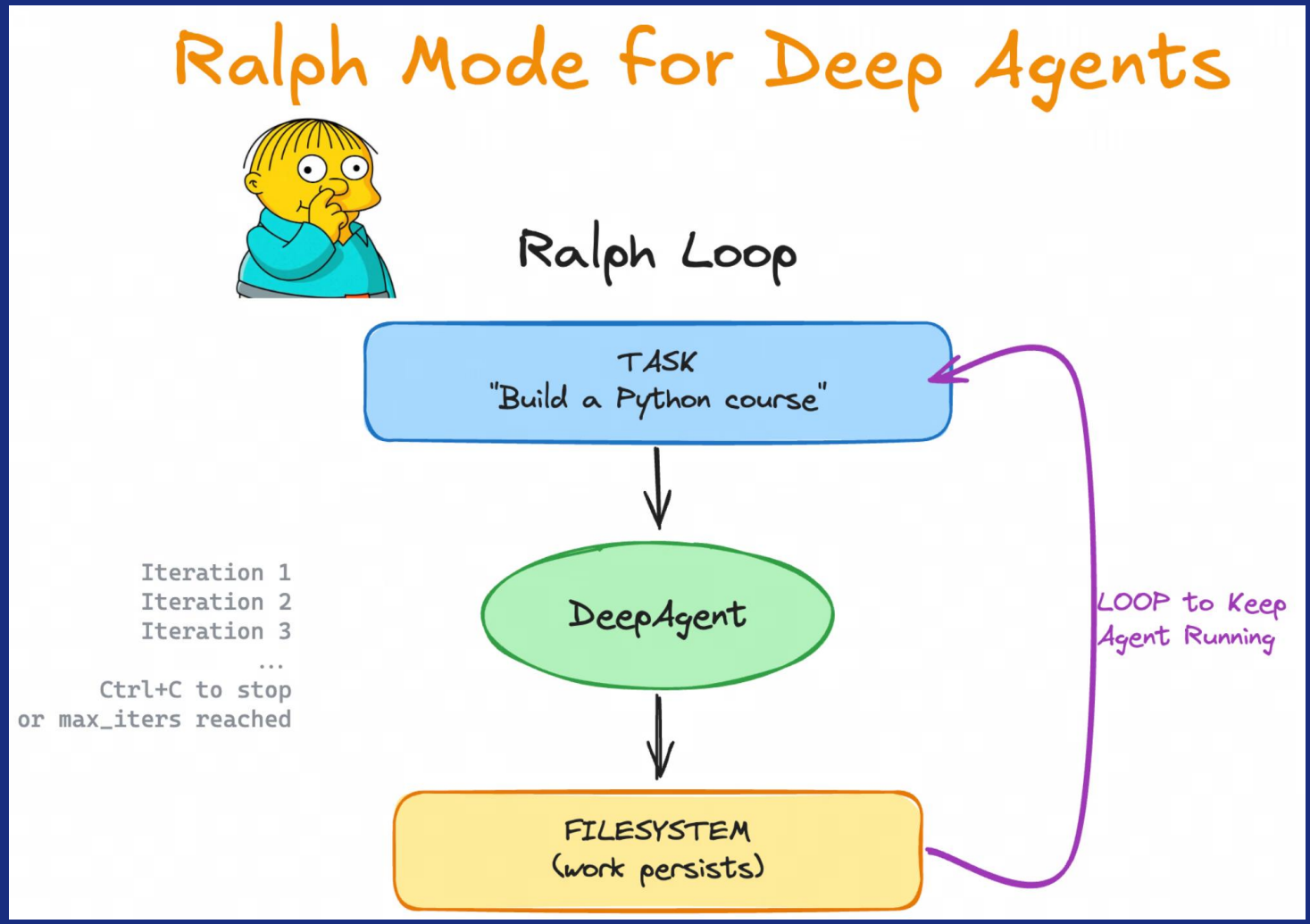
OMO – пример разделения ролей

Разные роли → разные когнитивные режимы → разные модели

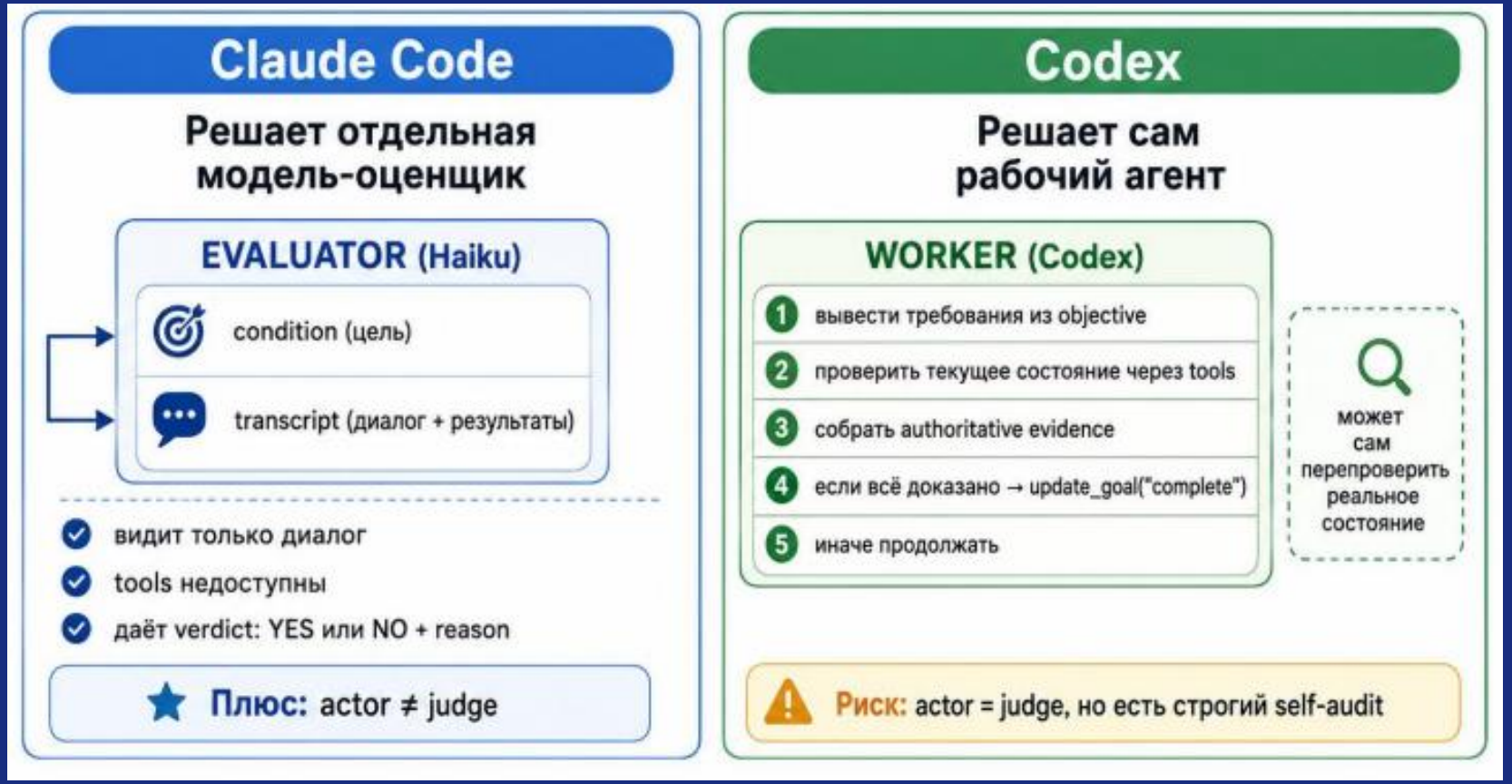
Роль	Главный вопрос	Режим мышления	Пример модели*
Sisyphus (Orchestrator)	Кого позвать и как собрать результат?	Оркестрация, декомпозиция	GPT-4o / Claude
Hephaestus (Deep Worker)	Как решить сложную техническую задачу?	Глубокое рассуждение, автономность	Claude Opus / o1
Explorer	Что есть в нашем коде?	Поиск, извлечение фактов	Claude Haiku / 3.5
Librarian	Что известно во внешнем мире?	Поиск, синтез знаний	GPT-4o / Perplexity
Oracle	Каков архитектурное решение лучше?	Анализ альтернатив, trade-offs	Claude Opus
Prometheus (Planner)	Что и в каком порядке делать?	Структурирование, планирование	GPT-4o
Metis (Gap Analyzer)	Что не учтено?	Критическое мышление	Claude 3.5
Momus (Critic)	План исполнен и проверяем?	Рецензирование, проверка	Claude 3.5 / o1
Atlas (Executor)	Обеспечить выполнение плана	Координация задач	GPT-4o
Sisyphus-Junior (Worker)	Выполнить задачу	Исполнение	Claude Haiku / 3.5

* Примерно. Выбор зависит от доступных моделей и задач.

/ralph-loop


Ralph Mode for Deep Agents
Ralph Loop
TASK
"Build a Python course"
DeepAgent
FILESYSTEM
(work persists)
LOOP to Keep
Agent Running
Iteration 1
Iteration 2
Iteration 3
...
Ctrl+C to stop
or max_iters reached

/goal


Claude Code
Решает отдельная модель-оценщик
EVALUATOR (Haiku)
condition (цель)
transcript (диалог + результаты)
видит только диалог
tools недоступны
даёт verdict: YES или NO + reason
Плюс: actor ≠ judge
Codex
Решает сам рабочий агент
WORKER (Codex)
1 вывести требования из objective
2 проверить текущее состояние через tools
3 собрать authoritative evidence
4 если всё доказано → update_goal("complete")
5 иначе продолжать
может сам перепроверить реальное состояние
Риск: actor = judge, но есть строгий self-audit

Пример /goal

```
/goal Use the nemo-rl-autoresearch and
nemo-rl-session-memory skill under `./skills`,
start a new auto research session, goal: Train
Qwen3-VL-2B-Instruct to high accuracy, using
SFT algorithm on the star count gym
environment. Start with profiling the model
out of the box accuracy. Time budget: 5h
```

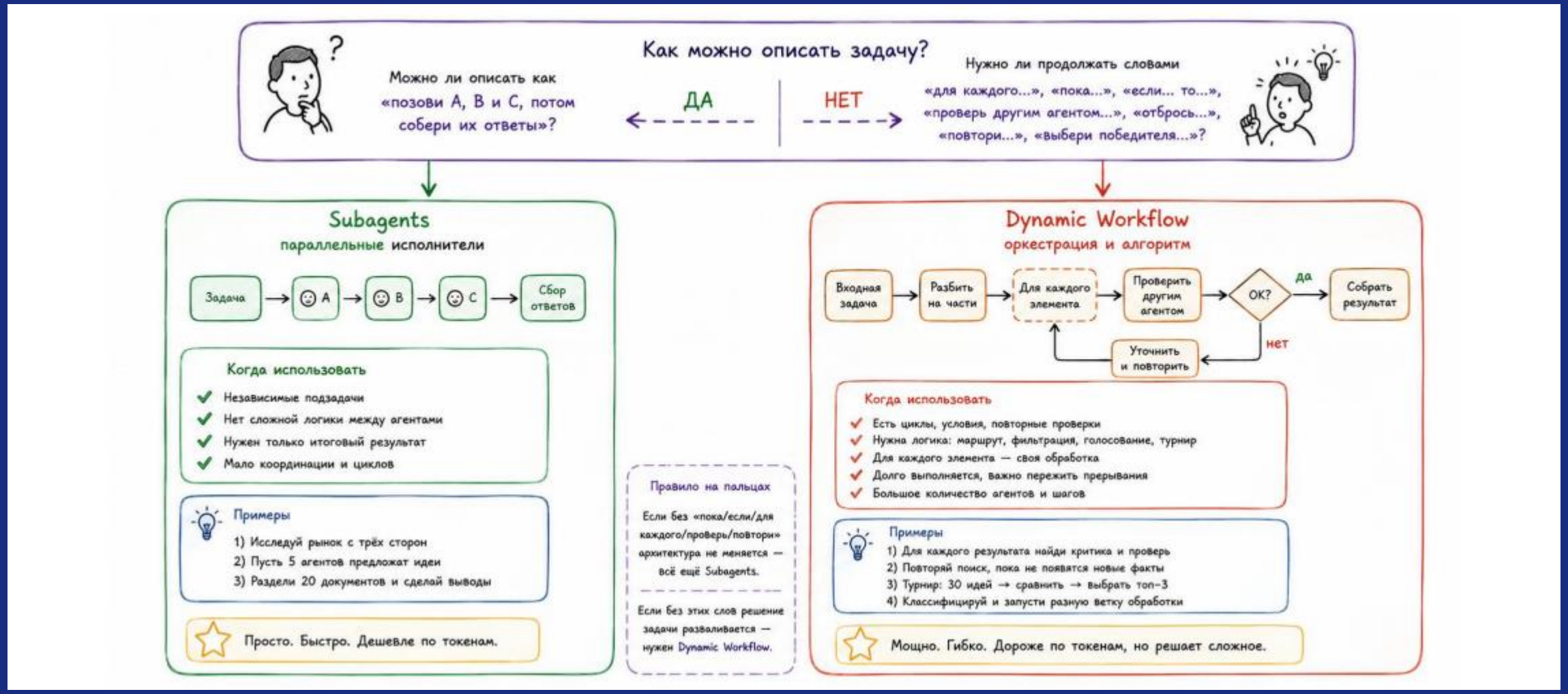
ВНИМАНИЕ

Сначала правильно задай цель и метрики, потом измеряй

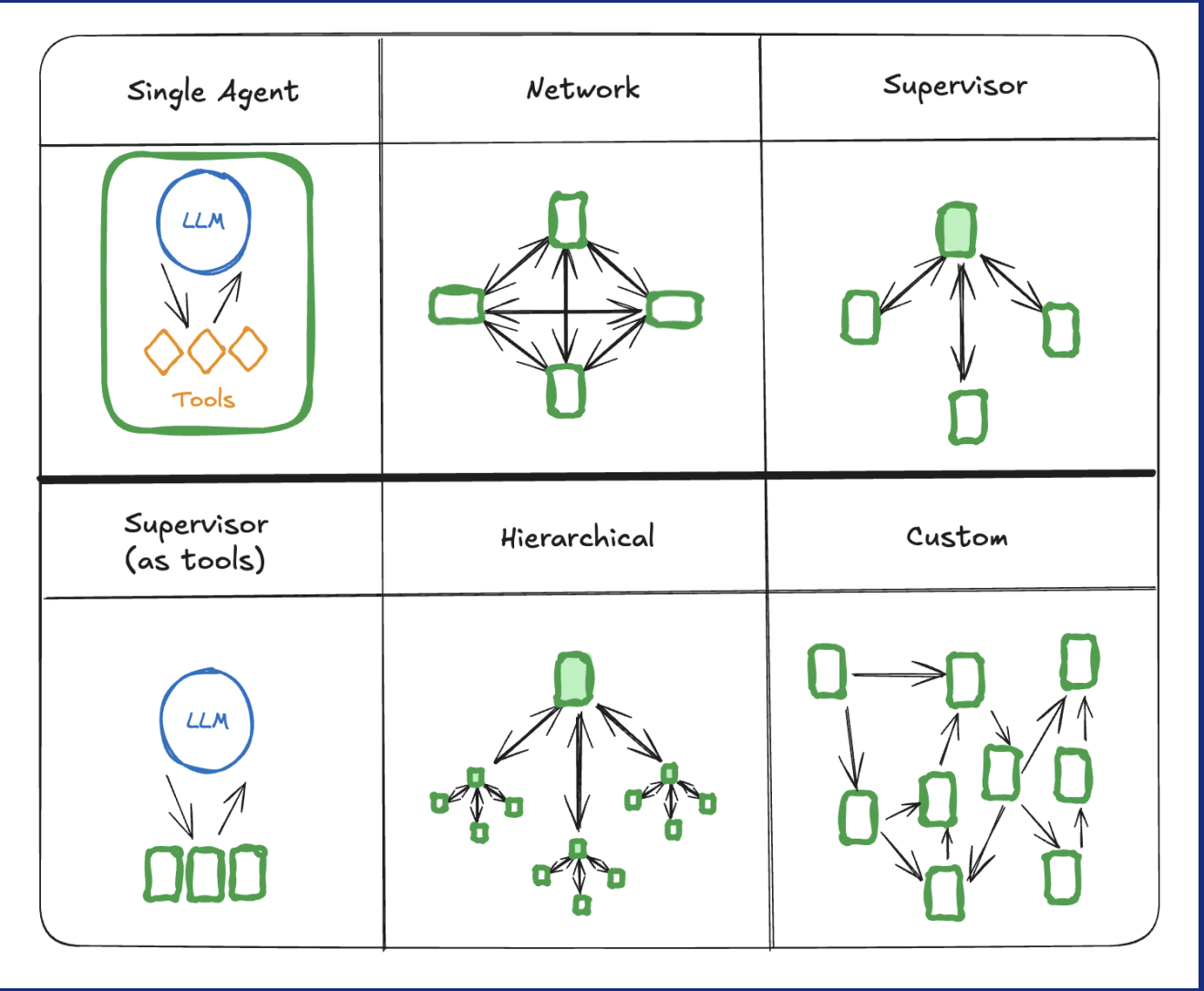
11:25



Делегирование



Графы агентов



ВНИМАНИЕ

Автономность ради автономности = отсутствие результата

11:25

